

JAVA 17 OCP · YDS STUDY SERIES

Unit 15 · JDBC — Practice Quiz

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu belge sekiz adet **OCF tarzı özgün çalışma sorusu** içerir; sorular gerçek sınavdan alınmamıştır. Önerilen süre 25–30 dakikadır; istersen 1–4 ve 5–8 olarak iki oturuma böl. JDBC sorularında Java compiler'ın kontrol ettiği type/checked-exception kurallarıyla driver veya database tarafından runtime'da kontrol edilen SQL kurallarını ayır.

Sorular

Soru 1

Odak: JDBC interface ve implementation ayrımı

Aşağıdaki programın çıktısı nedir?

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class JdbcTypes {
    public static void main(String[] args) {
        System.out.print(Connection.class.isInterface() + ":"
            + DriverManager.class.isInterface() + ":"
            + PreparedStatement.class.isInterface());
    }
}
```

- A. `true:false:true`
- B. `false:true:false`
- C. `true:true:true`
- D. Kod derlenmez.

Soru 2

Odak: Checked `SQLException`

Aşağıdaki class için hangisi doğrudur?

```
import java.sql.Connection;

public class TransactionStart {
    void begin(Connection connection) {
        connection.setAutoCommit(false);
    }
}
```

- A. Başarıyla derlenir; method `void` olduğu için exception bildirimi gerekmez.
- B. **Does not compile**; `setAutoCommit()` tarafından bildirilen checked `SQLException` handle veya declare edilmelidir.
- C. Derlenir; yalnız connection `null` ise runtime'da `SQLException` oluşur.
- D. Derlenir ve autocommit her durumda `false` olur.

Soru 3

Odak: Doğru execute method'unu yorumlama

`PreparedStatement.execute()` method'unun döndürdüğü `boolean` neyi gösterir?

- A. SQL statement'ın hatasız tamamlandığını
- B. Transaction'ın commit edildiğini
- C. İlk result'ın bir `ResultSet` olup olmadığını
- D. En az bir row'un etkilendiğini

Soru 4

Odak: Bind variable, cursor ve `INOUT`

Aşağıdaki ifadelerden **hangi ikisi doğrudur?**

- A. İlk bind variable index'i `0` dır.
- B. Yeni bir `ResultSet` üzerinde `next()` çağırılmadan getter kullanmak ilk row'u okur.
- C. `INOUT` parameter için input value set edilmeli ve output type `registerOutParameter()` ile kaydedilmelidir.
- D. JDBC bind parameter index'leri `1` den başlar.
- E. Java compiler SQL metnindeki placeholder sayısını setter çağrılarıyla karşılaştırır.

Soru 5

Odak: Transaction boundary ve portability

Autocommit `false` iken açık bir transaction bulunan `Connection` kapatılmadan önce portable ve açık davranış için ne yapılmalıdır?

- A. Hiçbir şey; `close()` bütün driver'larda transaction'ı kesin commit eder.
- B. Hiçbir şey; `close()` bütün driver'larda transaction'ı kesin rollback eder.
- C. Uygun iş kuralına göre açıkça `commit()` veya `rollback()` çağrılmalıdır.
- D. Bir savepoint oluşturmak transaction'ı otomatik tamamlar.

Soru 6

Odak: English → Turkish / YDS

Aşağıdaki cümleyi doğal Türkçeye çevir ve `as long as` yapısının anlamını belirt:

A prepared statement can be reused as long as values have been assigned to all required bind variables before execution.

Soru 7

Odak: SQL `NULL` ile Java `int` ayrımı

Aşağıdaki program Java 17'nin standart, bellekte çalışan `CachedRowSet` uygulamasını kullanır; haricî veritabanı veya sürücü gerekmez. **Tek doğru** çıktıyı seç.

```
import java.sql.Types;
import javax.sql.rowset.RowSetMetaDataImpl;
import javax.sql.rowset.RowSetProvider;

public class NullColumn {
    public static void main(String[] args) throws Exception {
        try (var rows = RowSetProvider.newFactory().createCachedRowSet()) {
            var metadata = new RowSetMetaDataImpl();
            metadata.setColumnCount(1);
            metadata.setColumnName(1, "amount");
            metadata.setColumnTypes(1, Types.INTEGER);
            rows.setMetaData(metadata);
            rows.moveToInsertRow();
            rows.updateNull(1);
            rows.insertRow();
            rows.moveToCurrentRow();
            rows.beforeFirst();
            rows.next();
            int amount = rows.getInt(1);
            System.out.print(amount + ":" + rows.wasNull());
        }
    }
}
```

- A. `null:true`
- B. `0:false`
- C. `0:true`
- D. `NullPointerException` oluşur.

Soru 8

Odak: Transaction zaman çizelgesi

Başlangıçta `balance = 100` olan tek kayıt vardır. Bağlantı ve kayıt noktaları geçerlidir; tüm çağrılar başarılı olur. Başka bağlantı değişiklik yapmaz. Aşağıdaki adımlar sırayla aynı bağlantıda uygulanır:

- 1 `setAutoCommit(false)`
- 2 Bakiyeyi 10 artıran bir `UPDATE`
- 3 `Savepoint p = connection.setSavepoint()`
- 4 Bakiyeyi 20 artıran bir `UPDATE`
- 5 `connection.rollback(p)`
- 6 `connection.commit()`

Kalıcılaştırılan bakiye kaçtır? **Tek seçenek seç.**

- A. 100
- B. 110
- C. 130
- D. Henüz bir değişiklik kalıcılaştırmamıştır.

Dil aktarımı: “Rolling back to the savepoint discards only the changes made after it.” cümlesinde `made after it` hangi ismi niteler? `it` neye döner?

Cevaplar ve açıklamalar

Soru 1 — A

- **A doğru:** `Connection` ve `PreparedStatement` JDBC interface'leridir; `DriverManager` ise JDK'nin sağladığı concrete class'tır. Çıktı `true:false:true` olur.
- **B yanlış:** Interface/class rolleri ters çevrilmiştir.
- **C yanlış:** `DriverManager` interface değildir.
- **D yanlış:** Type'ların tamamı Java 17 `java.sql` API'sinde bulunur ve code classpath'te derlenir.

Soru 2 — B

- **A yanlış:** Return type'ın `void` olması checked exception kuralını değiştirmez.
- **B doğru:** `Connection.setAutoCommit(boolean)`, `SQLException` declare eder; `begin()` bunu catch etmediği veya `throws SQLException` ile bildirmediği için kod derlenmez.
- **C yanlış:** `null` connection bir `NullPointerException` nedeni olurdu, fakat mevcut kod compilation aşamasını geçemez.
- **D yanlış:** Driver call başarısız olabilir; ayrıca checked exception önce ele alınmalıdır.

Soru 3 — C

- **A yanlış:** `false`, failure anlamına gelmez; update count veya no-result durumunu gösterebilir.
- **B yanlış:** `execute()` tek başına transaction commit sonucu bildirmez.
- **C doğru:** `true`, ilk result'ın `ResultSet` olduğunu; `false`, update count veya result bulunmadığını belirtir.
- **D yanlış:** Etkilenen row sayısı için uygun update method'u veya update count incelenir; boolean "en az bir row" testi değildir.

Soru 4 — C ve D

- **A yanlış:** JDBC parameter index'leri zero-based değil one-based'dir.
- **B yanlış:** Initial cursor first row'dan önce konumlanır; önce `next()` çağrılmalıdır.
- **C doğru:** `INOUT`, hem input setter hem output registration gerektirir.
- **D doğru:** İlk placeholder index'i `1` dir.
- **E yanlış:** SQL sıradan bir `String`dir; placeholder count ve SQL validity çoğunlukla driver/database tarafından runtime'da denetlenir.

Soru 5 — C

- **A yanlış:** Active transaction ile `close()` için portable JDBC contract bütün driver'larda commit garantisi vermez.
- **B yanlış:** Aynı şekilde bütün driver'larda rollback garantisi de yoktur.
- **C doğru:** Explicit `commit()` veya `rollback()` transaction'ın sonucunu açık, niyetli ve portable hâle getirir.
- **D yanlış:** Savepoint transaction içinde bir bookmark'tır; transaction'ı tamamlamaz.

Soru 6 — Örnek çeviri

“Çalıştırmadan önce gerekli bütün bağlama değişkenlerine değer atanmış olduğu sürece hazırlanmış bir deyim yeniden kullanılabilir.”

`as long as`, burada süre değil **koşul** bildirir ve “-dığı sürece” diye çevrilir. `can be reused` modal passive'dir; `have been assigned` ise present perfect passive yapısıdır. Önceki parameter value temizlenmediyse setter'ın her execution öncesinde yeniden çağırılması gerekmez.

Soru 7 — C

- **C doğru:** SQL `NULL`, `getInt()` ile okunduğunda primitive dönüş değeri `0` olur. Hemen sonraki `wasNull()`, son okunan SQL değerinin `NULL` olduğunu bildirir: başarıyla derlenir ve `0:true` yazdırır.
- **A yanlış:** Primitive `int`, Java `null` değerini tutamaz.
- **B yanlış:** `0`, hem gerçek sıfırdan hem SQL `NULL` değerinden gelebilir; ayrımı `wasNull()` yapar.
- **D yanlış:** Burada `Integer` unboxing yapılmıyor; `getInt()` API'si primitive değer döndürüyor.

Önce `next()` ile geçerli satıra gelindiğine de dikkat et. `wasNull()`, herhangi bir sütunun değil **son okunan sütunun** durumunu sorar.

Soru 8 — B

- **B doğru:** Değerler sırasıyla `100 → 110 → [p] → 130 → 110 → commit` olur. `rollback(p)`, `p` sonrasındaki 20 artışını geri alır; önceki 10 artışı kalır.
- **A yanlış:** `rollback(p)`, transaction'ın tamamına uygulanan parametresiz `rollback()` değildir.
- **C yanlış:** Kayıt noktasından sonraki artış geri alınmıştır.
- **D yanlış:** Son adım açık `commit()` çağrısıdır; kalan değişiklik kalıcılaşır.

Çeviri: “Kayıt noktasına geri dönmek, yalnız o noktadan sonra yapılan değişiklikleri geri alır.” `made after it, changes` ismini niteleyen kısaltılmış edilgen yan cümledir; `it`, savepoint'e gönderme yapar.

Bu soru JDBC transaction akışını ölçer; canlı bir sürücü davranışı gözlemine dayanmaz. İşlemlerin başarılı olduğu varsayımı soru kökünde verilmiştir.